INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

AI-BASED EXPENSE TRACKER WITH ANALYTICS AND INVESTMENT RECOMMENDATIONS

M. PREM KUMAR – 24623119

A. SUBASH – 24623130

K. DHANUSH RAJ – 24613106

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY CSE

IN

ARTIFICIAL INTELLIGENCE

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

BATCH NO. 11

ACADEMIC YEAR: 2024-28

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "AI-BASED EXPENSE TRACKER WITH ANALYTICS AND INVESTMENT RECOMMENDATIONS" is the Bonafide work of M. PREM KUMAR – 24623119, A. SUBASH – 24623130 and K. DHANUSH RAJ – 24613106, who carried out the project under my supervision. Certified further that, to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

Dr. K. ULAGA PRIYA, M.E., Ph.D.
Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS, Chennai

SIGNATURE OF THE SUPERVISOR

Dr. MEENAKSHI N, M.Tech.,Ph.D.
Assistant Professor
Department of AI & ML
School of Engineering
VISTAS, Chennai

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the mini project report entitled "**AI-BASED EXPENSE TRACKER WITH ANALYTICS AND INVESTMENT RECOMMENDATIONS**", submitted to Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning, is a record of original work carried out by us under the guidance of Dr. Meenakshi N., Assistant Professor, Department of Artificial Intelligence and Machine Learning. The work has not been submitted, in part or in full, for the award of any other degree or diploma.

Place: Chennai
Date: 30.10.2026

M. PREM KUMAR – 24623119

A. SUBASH – 24623130

K. DHANUSH RAJ – 24613106

ACKNOWLEDGEMENT

It is a great pleasure and a proud privilege to acknowledge the help and support we received from the many people who made this project a success. The infrastructural support and laboratory facilities provided by the institution have been a constant source of motivation, and we sincerely thank our FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH, for making them available.

We are grateful to our HEAD OF THE DEPARTMENT, Dr. K. ULAGA PRIYA, M.E., Ph.D., Department of Artificial Intelligence and Machine Learning, for the continuous support and encouragement extended to us throughout the project.

We extend our warmest thanks to our PROJECT GUIDE, Dr. MEENAKSHI N, M.Tech., Ph.D., Assistant Professor, Department of Artificial Intelligence and Machine Learning, who patiently guided us and provided invaluable advice at every stage of this work.

We also express our gratitude to the Project Coordinator Ms. F. Benasir Begam M.E., (PhD) faculty members, the librarian and the non-teaching staff of the department, who helped and supported us throughout. Last but not least, we thank our parents and friends for their support and encouragement, without which this project would not have been possible.

M. PREM KUMAR – 24623119
A. SUBASH – 24623130
K. DHANUSH RAJ – 24613106

TABLE OF CONTENTS

TITLE **PAGE NO.**

ABSTRACT i
LIST OF FIGURES AND TABLES ii
LIST OF ABBREVIATIONS iii
CHAPTER 1. INTRODUCTION **1**
1.1 OVERVIEW OF THE PROJECT 1
1.2 PROBLEM STATEMENT 2
1.3 OBJECTIVES 2
1.4 SCOPE OF THE PROJECT 2
CHAPTER 2. LITERATURE REVIEW **4**
2.1 LITERATURE OVERVIEW 4
2.2 LITERATURE SURVEY 4
2.3 COMPARATIVE LITERATURE REVIEW 6
2.4 RESEARCH GAP 6
CHAPTER 3. SYSTEM DESIGN **7**
3.1 EXISTING SYSTEM 7
3.2 PROPOSED SYSTEM 7
3.3 SYSTEM ARCHITECTURE 8
3.4 SYSTEM WORKFLOW 9
3.5 DATA FLOW DIAGRAMS 10
3.6 SYSTEM REQUIREMENTS 11
CHAPTER 4. IMPLEMENTATION **13**
4.1 METHODOLOGY 13
4.2 SYSTEM MODULES 13
4.3 FRONTEND AND BACKEND IMPLEMENTATION 14
4.4 DATABASE DESIGN 15
4.5 DATA VALIDATION (SMART DATA VERIFICATION) 16
4.6 FINANCIAL ANALYTICS AND RECOMMENDATIONS 18
4.7 TESTING AND RESULTS 21
4.8 SCREENSHOTS 22
CHAPTER 5. CONCLUSION AND FUTURE WORK **25**
5.1 CONCLUSION 25
5.2 LIMITATIONS 26
5.3 FUTURE ENHANCEMENTS 26
REFERENCES **27**
APPENDIX A. CONFERENCE PAPER **29**
APPENDIX B. SOURCE CODE **34**

ABSTRACT

Managing personal finances requires individuals to monitor their income, expenses, savings, bills and financial goals. Notebooks and spreadsheets require manual calculation and give little support for spotting spending patterns or planning ahead. This project, AI-Based Expense Tracker with Analytics and Investment Recommendations, presents a web-based personal finance application that brings these activities together on one platform.

Users enter income and expenses manually. Each entry is checked by a Smart Data Verification step, reviewed and confirmed by the user, and validated again by the server before it is saved. The stored transactions drive a dashboard of totals and category-wise and month-wise charts, together with expense prediction, budget recommendations, a Financial Goal Planner with feasibility analysis, saving suggestions, an educational investment guide, bill reminders, a 100-point Financial Health Score and a downloadable monthly PDF report.

The system uses React.js for the frontend, Python Flask with SQLAlchemy for the backend and SQLite for storage. Login uses hashed passwords and signed tokens, and each user can access only his or her own records. The analytics are transparent, rule-based calculations on the user's own data rather than a trained machine-learning model, so every recommendation can be explained. Their usefulness depends on the completeness of the data entered, and investment guidance is for educational purposes only.

Keywords: *Expense tracking, personal finance management, financial analytics, expense prediction, budget recommendation, goal-based saving, financial health score, React.js, Flask, SQLite.*

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	System architecture of the AI Expense Tracker	9
Figure 3.2	System workflow of the AI Expense Tracker	10
Figure 3.3	Level 0 data flow diagram	10
Figure 3.4	Level 1 data flow diagram	11
Figure 4.1	Database design (entity–relationship diagram)	15
Figure 4.2	Smart Data Verification flow	17
Figure 4.3	Dashboard with summary cards and charts	22
Figure 4.4	Financial Goal Planner with feasibility analysis	23
Figure 4.5	Bill Reminder module	23
Figure 4.6	Financial Health Score	23
Figure 4.7	Saving Suggestions	24
Figure 4.8	Login page	24

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 2.1	Comparative literature review	6
Table 3.1	Hardware and software requirements	11
Table 4.1	System modules and where they are implemented	13
Table 4.2	Main REST API endpoints	14
Table 4.3	Database tables	16
Table 4.4	Validation rules	17
Table 4.5	Financial Health Score factors	20
Table 4.6	Test results	21

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DT	Decision Tree
FD / RD	Fixed Deposit / Recurring Deposit
FK / PK	Foreign Key / Primary Key
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbors
LSTM	Long Short-Term Memory
ML	Machine Learning
MLP	Multi-Layer Perceptron
NLP	Natural Language Processing
OCR	Optical Character Recognition
PDF	Portable Document Format
REST	Representational State Transfer
RF	Random Forest
RL	Reinforcement Learning
SQL	Structured Query Language
SVM	Support Vector Machine

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Managing personal finances means keeping track of income, expenses, bills and savings for future plans such as buying a vehicle or a laptop. Many people still record this information in notebooks, scattered mobile notes or spreadsheets. These methods require repeated manual calculation, make spending patterns hard to see and give little help in planning ahead.

The **AI-Based Expense Tracker with Analytics and Investment Recommendations** (the *AI Expense Tracker*) is a web-based personal finance application that brings these activities together on one platform. The user records income and expenses manually. Each entry passes through a **Smart Data Verification** step, where it is checked and shown back to the user for confirmation before it is saved. The stored transactions are then used to calculate total income, total expenses and savings, and to present category-wise and month-wise analytics on a dashboard.

On top of this transaction base, the application provides expense prediction, budget recommendations, a Financial Goal Planner with feasibility analysis, saving suggestions, educational investment guidance, bill reminders, a Financial Health Score and a downloadable monthly PDF report. It is built with **React.js** for the frontend, **Python Flask** with **SQLAlchemy** for the backend and **SQLite** for storage.

The "AI" in the title refers to the intelligent assistance the application gives the user. In the current version this assistance is implemented with **transparent, rule-based calculations** on the user's own data, not with a trained machine-learning model, so every result can be explained to the user. Trained models are identified as future work.

1.2 PROBLEM STATEMENT

Many individuals find it difficult to keep accurate records of daily income and expenses and to understand how their spending affects their savings and goals. Manual methods are slow and error-prone, and they do not give a combined view of income, expenses and balance. Basic expense trackers mostly stop at record-keeping and offer little support for predicting expenses, recommending budgets, planning goals, tracking bills or assessing overall financial health. An integrated application is therefore needed that records verified transactions and turns them into clear, practical financial guidance.

1.3 OBJECTIVES

1. To develop a centralized web application for recording and managing income and expenses.
2. To validate and confirm transactions before saving, reducing incomplete or invalid records.
3. To calculate income, expenses, savings and balance automatically and present category-wise and month-wise analytics.
4. To provide expense prediction and budget recommendations from the recorded data.
5. To support financial goal planning with progress tracking and feasibility analysis.
6. To offer saving suggestions, educational investment guidance and bill reminders.
7. To assess overall financial condition through a Financial Health Score and a monthly PDF report.

1.4 SCOPE OF THE PROJECT

The current version covers account registration and sign-in; adding, viewing, editing and deleting transactions; dashboard analytics; expense prediction and

budget recommendations; goals; bills; saving suggestions; the investment guide; the Financial Health Score; and the monthly PDF report. Each user can see and change only his or her own records.

The following are **not** part of the current version and are treated as future enhancements: connection to bank accounts or automatic import of statements, voice-based entry, photo-based bill scanning (OCR), prediction with a trained machine-learning model, and personalized or regulated investment advice.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

Research on personal finance management has moved from simple digital ledgers towards intelligent systems that categorize transactions, forecast spending and recommend actions. Recent studies use supervised machine-learning algorithms (Decision Tree, Random Forest, SVM, XGBoost) for categorization and budget prediction, LSTM networks for time-series forecasting, NLP models such as BERT for transaction descriptions, and OCR for reading receipts. This chapter reviews five recent papers studied by the team as base papers for the project and identifies the research gap. The methods, datasets, results and limitations below are summarized from the team's review of these papers.

2.2 LITERATURE SURVEY

[1] Chahar, Vishwakarma, Mishra and Paliwal (2025) propose an *Artificial Intelligence Powered Personal Finance Management System* for personalized budgeting, expense classification, forecasting and recommendations. Their modular architecture combines an expense classification engine (Random Forest, SVM, LSTM), predictive analytics, a recommendation system and an NLP interface, built with React.js, Flask, MongoDB and Firebase. The system improved financial decision-making and achieved a user-satisfaction rating of 4.4 out of 5. Its limitations are weak adaptability to irregular income and spending, heavy dependence on data quality, and unresolved trust and regulatory barriers. This paper is closest to our React.js and Flask design; our project adds goal feasibility, bill reminders and an explainable health score.

[2] Jadhav and Patil (2025) present an *AI-ML Based Expense Categorization and Budgeting System* that uses a Hybrid Machine Learning algorithm (Decision Tree, SVM and Random Forest) with supervised and unsupervised

learning, NLP and reinforcement learning on a real-life financial dataset. The system reported about 97% recommendation accuracy. However, it lacks deep-learning baselines and does not address the explainability of its decisions. This explainability gap motivates our rule-based approach, where each suggestion states the figures behind it.

[3] Kumar, Vishwakarma, Sainy, Kumar and Aggarwal (2024) develop a *Personal Finance Manager with Predictive Analytics Using LSTM*. LSTM forecasting was applied to a synthetic dataset of 200 users and 2,400 records, with a web interface using Chart.js. The system forecasts income and expenses and flags overspending, but it needs long histories, struggles with irregular events and lacks personalized goal-setting. This motivates our Goal Planner, which compares each goal's monthly requirement with the user's actual surplus.

[4] Thakur and Jadhav (2025) compare XGBoost, Random Forest, SVM, MLP, KNN, Decision Tree and CatBoost, with ensemble techniques, in an *Expense Tracker Management System Using Machine Learning* built with Django and PostgreSQL. XGBoost performed best individually and a Voting Ensemble reached an R^2 of 78.11% on a household transactions dataset. Results plateaued on the single dataset, limiting generalization. Since a new user of our application has little history, we use transparent calculations and plan trained models for later.

[5] Saranya, D'Souza and Fahadhu (2026) describe an *AI-Powered Intelligent Expense Tracking and Predictive Financial Management System* that uses BERT for categorization, LSTM for forecasting and OCR for receipts, reporting above 94% categorization accuracy and about ±6% forecast variation. The approach depends on OCR accuracy and on access to sensitive bank and wallet data, which raises privacy concerns. This supports our choice of manual, user-controlled entry, with OCR scanning left as a future enhancement.

2.3 COMPARATIVE LITERATURE REVIEW

Table 2.1: Comparative literature review

Ref.	Method / Algorithm	Data Used	Reported Result	Limitation
[1]	RF, SVM, LSTM; recommendation system; NLP	Transactions, user profiles, budget data	User satisfaction 4.4/5	Weak on irregular income; trust and compliance
[2]	Hybrid ML (DT + SVM + RF), NLP, RL	Real-life financial dataset	~97% recommendation accuracy	No deep-learning baselines; not explainable
[3]	LSTM forecasting; Chart.js dashboard	Synthetic: 200 users, 2,400 records	Forecasts income and expenses	Needs long history; weak on irregular events
[4]	XGBoost, RF, SVM, MLP, KNN, DT, CatBoost; ensembles	Household transactions dataset	Voting Ensemble R^2 78.11%	Single dataset; limited generalization
[5]	BERT, LSTM, OCR	Receipts, bills, bank and wallet data	>94% categorization accuracy	OCR dependence; privacy risk

2.4 RESEARCH GAP

- **Limited integration:** Most systems focus on recording and categorizing transactions, with little integration of prediction, budgeting, goal planning and guidance in one application.
- **Weak link between spending and goals:** Few systems check whether a goal is achievable from the user's actual monthly surplus.
- **Explainability:** Model-driven systems rarely explain why a recommendation was made.
- **Privacy and data needs:** The strongest results depend on large or bank-linked datasets that a new, privacy-conscious user cannot provide.

The AI Expense Tracker addresses these gaps with an integrated, user-controlled application in which verified manual entries drive transparent analytics, goal feasibility analysis, saving suggestions, bill reminders and a Financial Health Score.

CHAPTER 3. SYSTEM DESIGN

3.1 EXISTING SYSTEM

People currently manage their money with notebooks, spreadsheets, basic expense-tracking apps or the transaction history of their banking apps. These methods have clear limitations:

- **Manual calculation:** Totals, savings and balances must be worked out by hand or with user-built formulas, which invites mistakes.
- **No verification:** Missing amounts, invalid dates or wrong categories are not checked before being recorded.
- **Scattered information:** Income, expenses, goals and bills are kept in different places, so there is no single view of the financial position.
- **No forward-looking guidance:** Existing methods rarely estimate future expenses, recommend budgets or check whether a goal can be reached by its deadline.
- **Missed bills and no health indicator:** Due dates are easily forgotten, and users have no simple measure of whether their habits are improving.

3.2 PROPOSED SYSTEM

The proposed AI Expense Tracker is a web application that combines verified transaction management with rule-based analytics and guidance. The user enters data through React forms. Entries are checked in the browser, reviewed and confirmed by the user, and then sent to the Flask backend, which validates them again and stores them in SQLite. The frontend loads the user's records and calculates summaries, predictions, goal feasibility, suggestions, bill status and the Health Score, which are shown on the dashboard and its tabs. The monthly PDF report is generated by the backend.

Compared with the existing system, it offers automatic calculations, verification before saving, a single dashboard, forward-looking guidance (prediction, budgets, goal feasibility), bill reminders and an explainable health score, without requiring any bank connection.

3.3 SYSTEM ARCHITECTURE

The system follows a three-tier client–server architecture (Figure 3.1):

- **Presentation layer (React.js):** Renders the interface, collects and checks input, calls the backend through a single API helper (api.js) and performs all financial analytics in a dedicated module (finance.js). Charts are drawn with Recharts [6], [10].
- **Application layer (Python Flask):** Exposes REST API endpoints, issues and checks signed login tokens, validates every request, stores records through Flask-SQLAlchemy and builds the monthly PDF with ReportLab [7], [9], [11].
- **Data layer (SQLite):** Stores users, transactions, goals, bills and saving-suggestion states in a single file-based database [8].

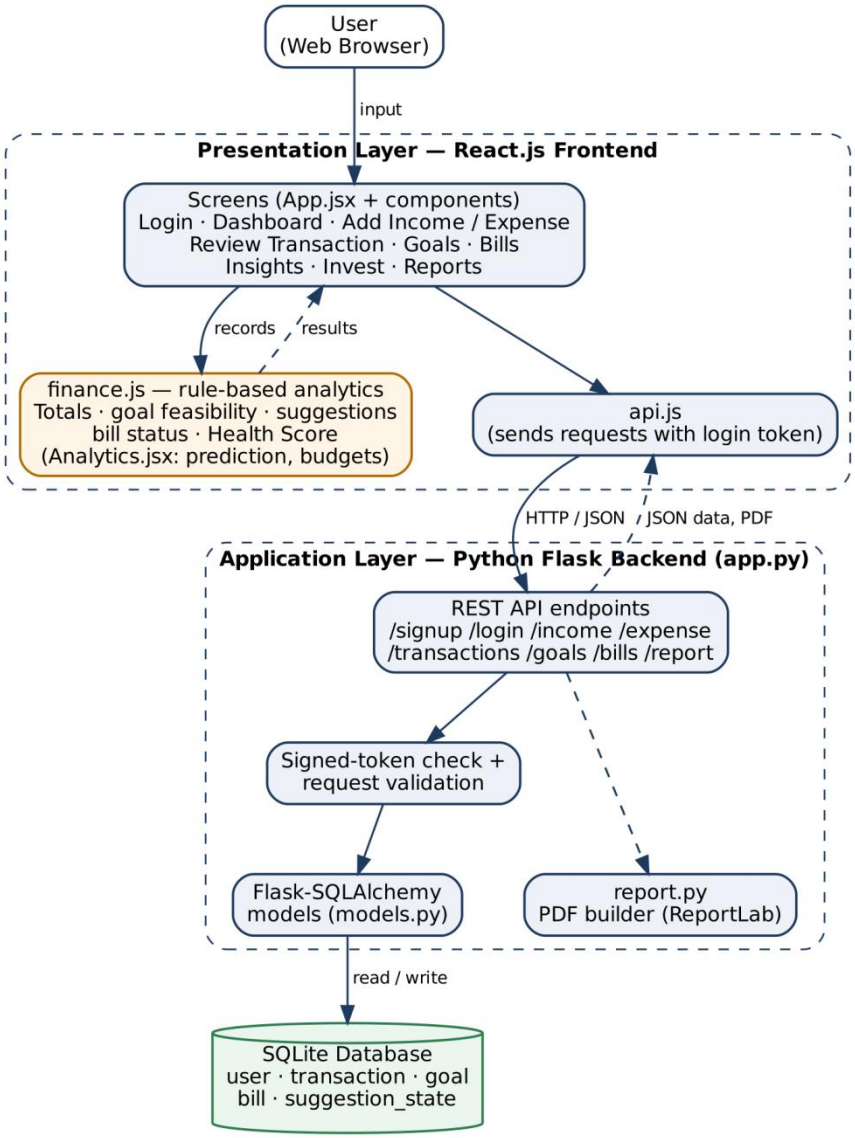


Figure 3.1: System architecture of the AI Expense Tracker

3.4 SYSTEM WORKFLOW

Figure 3.2 shows the end-to-end workflow designed by the team. The user signs in and reaches the dashboard. Financial data passes through Smart Data Verification before backend processing. Confirmed transactions are saved in SQLite; edits and deletions update the stored records. The stored data feeds spending analysis, prediction and budgeting, financial planning and financial monitoring, whose results are displayed on the dashboard, and the selected month's data is compiled into the monthly report.

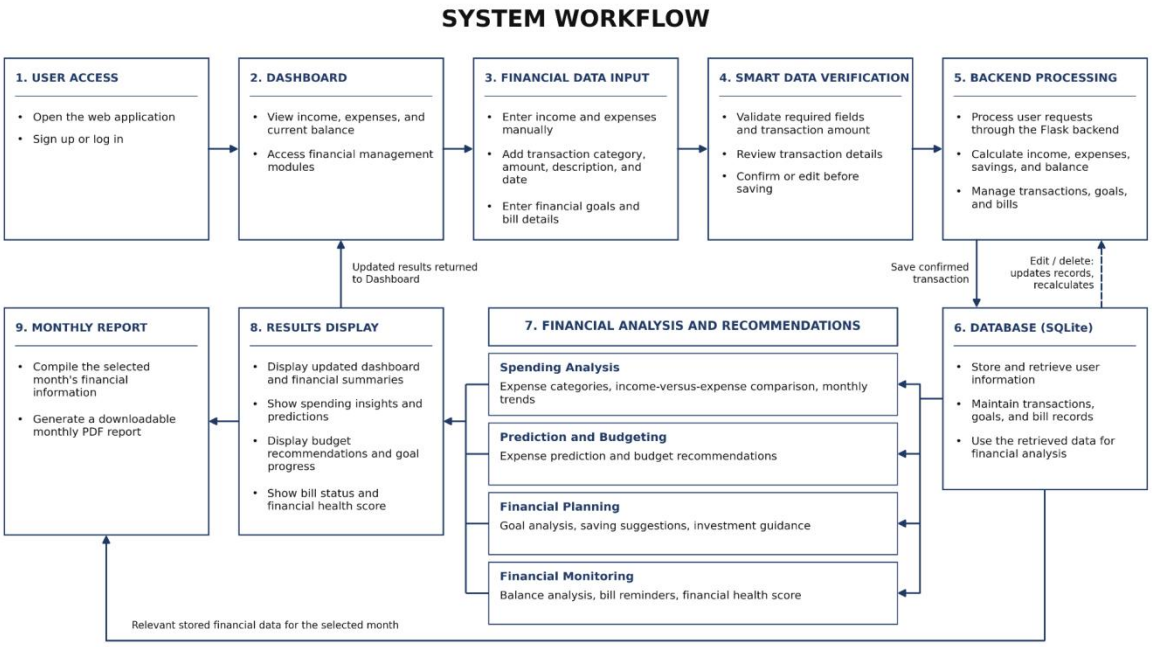


Figure 3.2: System workflow of the AI Expense Tracker

3.5 DATA FLOW DIAGRAMS

The Level 0 diagram (Figure 3.3) treats the application as a single process that exchanges data with the user. The Level 1 diagram (Figure 3.4) breaks it into eight processes and four main data stores: authentication uses the Users store, verified transactions are written to the Transactions store, and the analysis processes read from the stores to produce summaries, predictions, goal feasibility, bill status, the Health Score and the monthly report.

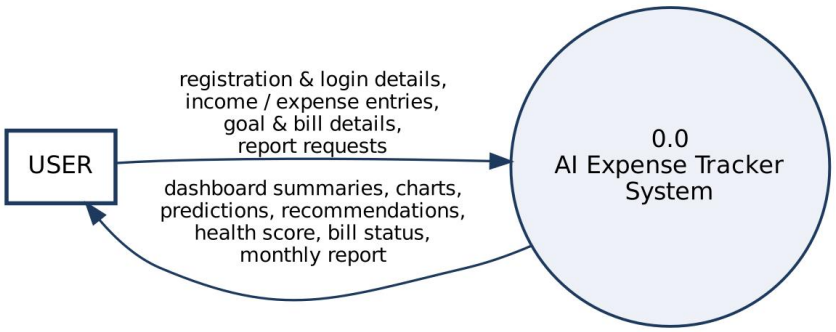


Figure 3.3: Level 0 data flow diagram

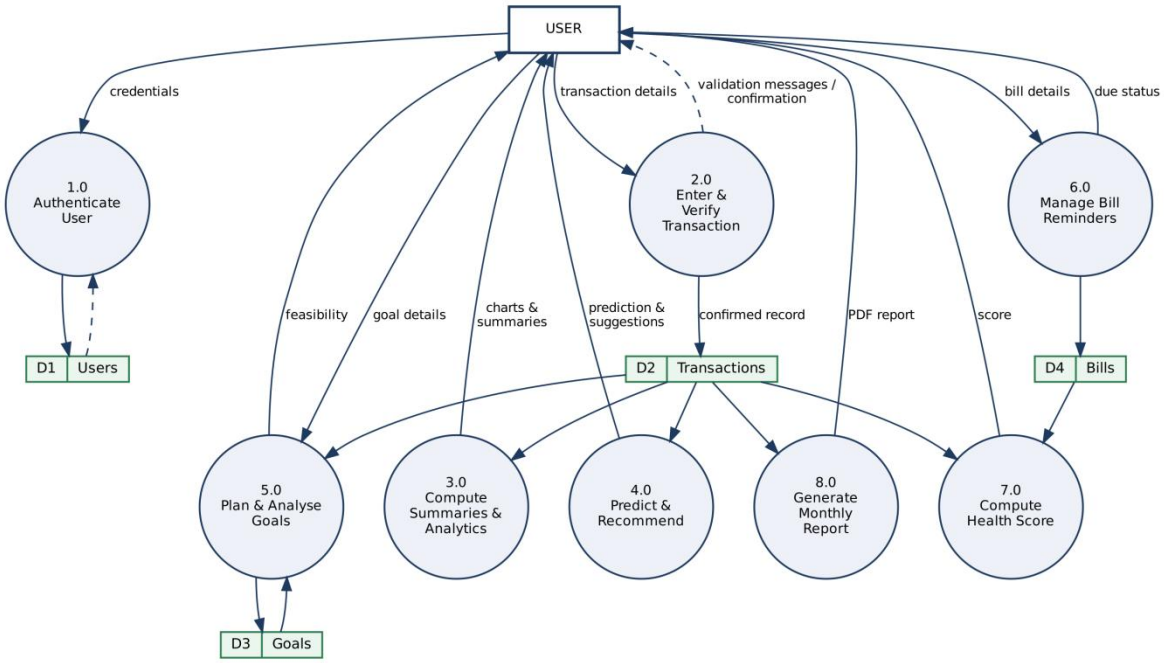


Figure 3.4: Level 1 data flow diagram

3.6 SYSTEM REQUIREMENTS

Table 3.1: Hardware and software requirements

Item	Specification
Processor	Intel Core i3 or above
RAM	Minimum 4 GB (8 GB recommended)
Storage	Minimum 10 GB free space
Operating system	Windows 10 / 11
Frontend	React.js 19, JavaScript, HTML, CSS, Recharts, Vite
Backend	Python, Flask, Flask-SQLAlchemy, Flask-CORS, ReportLab
Database	SQLite
Tools	Visual Studio Code, Node.js and npm, web browser

The application performs lightweight calculations on the user's own records and does not train machine-learning models, so an ordinary laptop is sufficient.

Functional requirements: user registration and sign-in; add, edit and delete income and expense transactions with verification; dashboard totals and charts; expense prediction and budget recommendations; goals with feasibility analysis;

bill reminders; saving suggestions; investment guidance; Health Score; monthly PDF report.

Non-functional requirements: each user accesses only his or her own data; passwords are never stored in plain text; invalid records are rejected; recommendations state the figures behind them; investment guidance is clearly marked as educational; the interface is simple enough for users without financial training.

CHAPTER 4. IMPLEMENTATION

4.1 METHODOLOGY

The project followed an incremental, modular approach over twelve weeks. Problem identification, the literature survey and the system and database design came first. The React frontend with authentication and the Flask backend with SQLite were built next, followed by core transaction management and Smart Data Verification, because every analytical feature depends on reliable transaction data. The analytical modules (prediction and budgets, goals, suggestions, investment guidance, bills, Health Score and the PDF report) were then added one at a time, each integrated with the dashboard and tested before the next was started. The final weeks were used for interface refinement, testing and documentation.

All analytics are **deterministic, rule-based calculations** on the user's own records, chosen for three reasons: a new user has too little history to train a reliable model [3], [4]; each result can be explained to the user [2]; and no external dataset or bank connection is needed. No machine-learning model was trained and no prediction accuracy is claimed.

4.2 SYSTEM MODULES

Table 4.1: System modules and where they are implemented

Module	Main Function	Implemented In
Authentication	Sign up, sign in with signed token, sign out	app.py, App.jsx
Transactions	Add, list, edit and delete income and expenses	app.py, TransactionList.jsx
Smart Data Verification	Check, review and confirm before saving	App.jsx, app.py
Dashboard and analytics	Totals, category and monthly charts	Analytics.jsx, finance.js
Prediction and budgets	Next-month estimate; budget limits	Analytics.jsx
Goal Planner	Goals, progress, feasibility analysis	GoalPlanner.jsx, finance.js

Module	Main Function	Implemented In
Bill Reminder	Bills, due status, mark as paid	BillReminder.jsx, app.py
Saving Suggestions	Rule-based savings opportunities	finance.js
Investment Guide	Educational comparison of saving options	InvestmentGuide.jsx
Financial Health Score	100-point score with factor breakdown	finance.js
Monthly Report	Downloadable PDF for a chosen month	report.py

4.3 FRONTEND AND BACKEND IMPLEMENTATION

Frontend. The React application (App.jsx) shows the sign-in and registration screen, then a header with *Add Income* and *Add Expense* buttons and tabs for the dashboard, goals, bills, insights, investments and reports. All requests go through api.js, which attaches the login token and turns server errors into readable messages. All money calculations live in one plain JavaScript module, finance.js. Every screen calls it with the same list of transactions, so totals, goals, suggestions and the Health Score always agree after a record is added, edited or deleted. Amounts are shown in Indian Rupees with Indian digit grouping.

Backend. The Flask application (app.py) exposes the REST endpoints in Table 4.2. When a user signs in, the server returns a token signed with a secret key and valid for seven days. Protected routes read the user's identity from this token, never from the request body, and every query is filtered by that user's id, so one user can never read or change another user's data. Passwords are stored as salted hashes using Werkzeug.

Table 4.2: Main REST API endpoints

Endpoint	Method	Purpose
/signup, /login, /me	POST, POST, GET	Create account; sign in and receive token; check saved login

Endpoint	Method	Purpose
/income, /expense	POST	Add a validated transaction
/transactions	GET	List the user's transactions, newest first
/transactions/{id}	PUT, DELETE	Edit or delete one of the user's transactions
/goals, /goals/{id}	GET, POST, PUT, DELETE	Manage savings goals
/bills, /bills/{id}, /bills/{id}/pay	GET, POST, PUT, DELETE, POST	Manage bills; mark a bill as paid
/suggestions/state	GET, PUT	Remember completed or dismissed suggestions
/report/monthly	GET	Download the monthly PDF report

4.4 DATABASE DESIGN

The SQLite database is defined with Flask-SQLAlchemy models (models.py) and created automatically when the server starts. Each user owns many transactions, goals, bills and suggestion states (Figure 4.1).

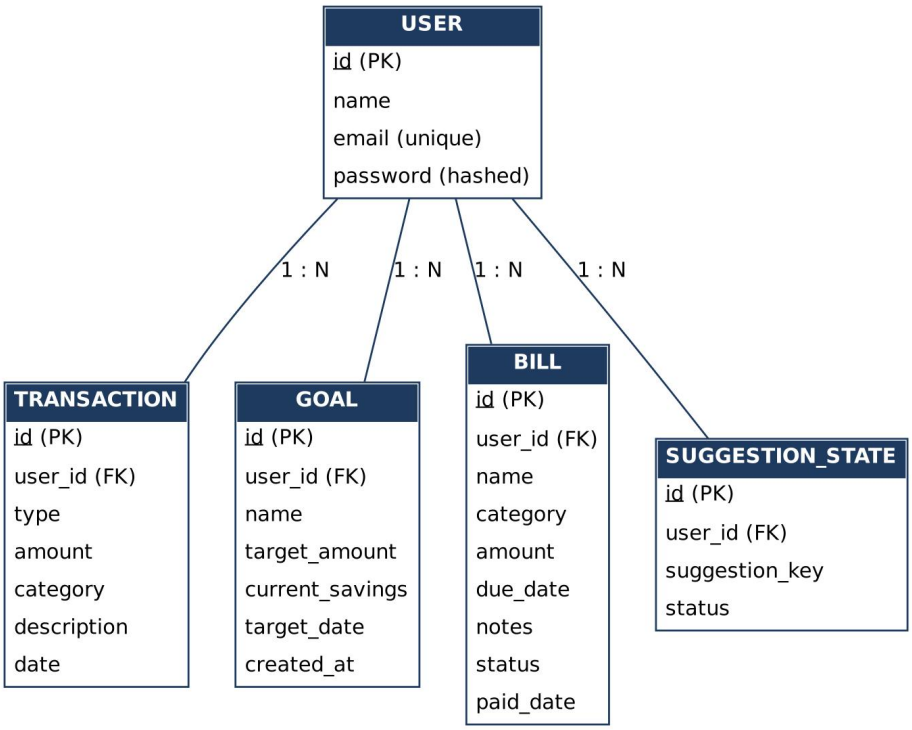


Figure 4.1: Database design (entity–relationship diagram)

Table 4.3: Database tables

Table	Fields
user	id (PK), name, email (unique), password (hashed)
transaction	id (PK), user_id (FK), type (income / expense), amount, category, description, date
goal	id (PK), user_id (FK), name, target_amount, current_savings, target_date, created_at
bill	id (PK), user_id (FK), name, category, amount, due_date, notes, status (paid / unpaid), paid_date
suggestion_state	id (PK), user_id (FK), suggestion_key, status (completed / dismissed)

4.5 DATA VALIDATION (SMART DATA VERIFICATION)

Every analysis depends on the entered data, so each transaction is checked twice before it is stored (Figure 4.2, Table 4.4). The form first checks the entry in the browser and warns the user about unusually large expenses. A *Review Transaction* page then shows the details, and the user chooses *Confirm & Save* or *Edit Details*. The backend validates the request again, so invalid data is rejected even if the browser checks are bypassed.

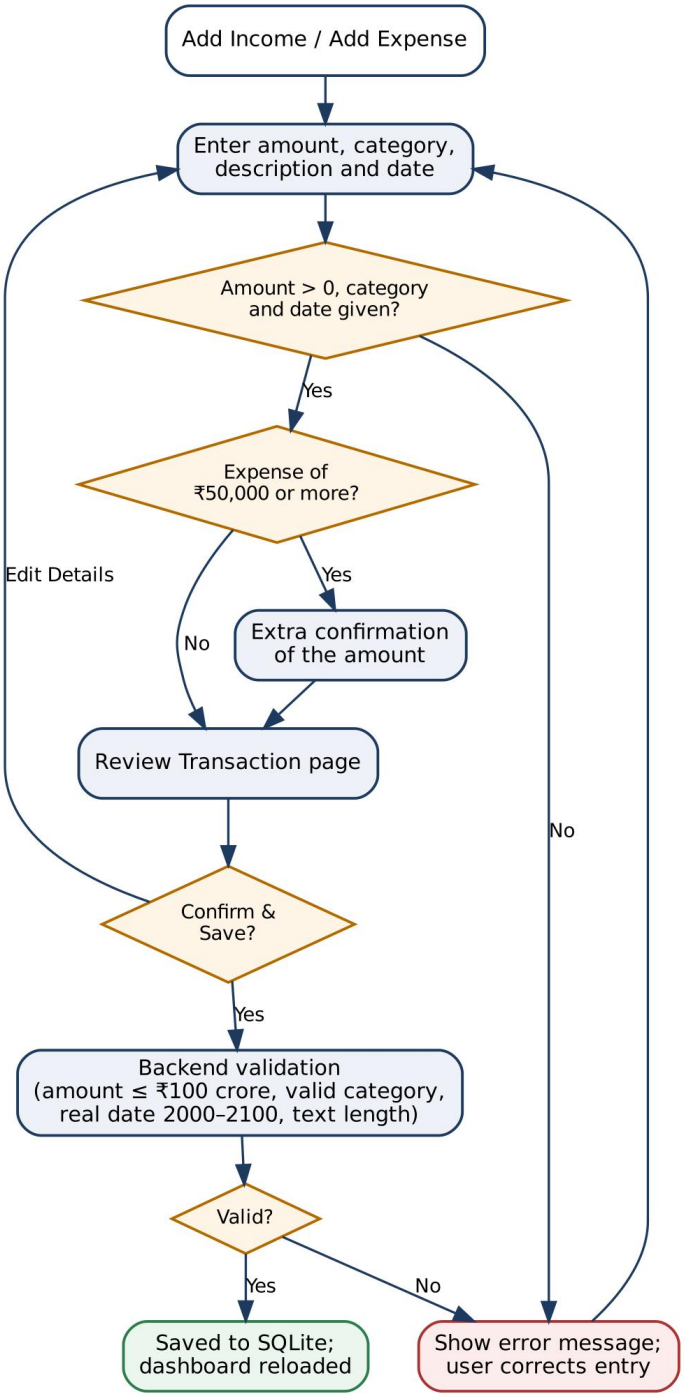


Figure 4.2: Smart Data Verification flow

Table 4.4: Validation rules

Check	Rule (as implemented)
Amount	Required, numeric, greater than ₹0 and at most ₹100 crore
Category	Must be selected from the fixed income or expense category list

Check	Rule (as implemented)
Date	Required; a real calendar date (YYYY-MM-DD) between 2000 and 2100
Description	Optional; at most 200 characters
High-value expense	Expenses of ₹50,000 or more need an extra confirmation
Review and confirm	Nothing is saved until the user clicks *Confirm & Save*
Account fields	Valid email format; password of at least 6 characters; unique email

4.6 FINANCIAL ANALYTICS AND RECOMMENDATIONS

4.6.1 Core Calculations

$$Total\ Savings\ (Balance) = Total\ Income - Total\ Expenses$$

$$Savings\ Rate\ (\%) = (Total\ Income - Total\ Expenses) \div Total\ Income \times 100$$

Estimates that need a monthly average use the last six calendar months that contain transactions; months with no records are not counted, so no zero months are invented.

4.6.2 Expense Prediction and Budget Recommendation

With one month of expense data, the next month's expense is estimated as the average monthly expense. With two or more months, it continues the latest trend:

$$Predicted\ Expense = Last\ Month + (Last\ Month - Previous\ Month),\ never\ below\ 0$$

For budget recommendations, each expense category is compared with total income. A category taking 25% or more of income is given a recommended limit 20% below its current spending, and a category taking 15–25% is given a limit 10% lower. Other categories keep their current amount.

4.6.3 Goal Planner and Feasibility Analysis

$$Progress\ (\%) = Current\ Savings \div Target\ Amount \times 100$$

$$Monthly\ Saving\ Required = (Target - Saved) \div Months\ Remaining$$

Months remaining are rounded up from the days left (at least one month). The feasibility analysis compares the requirement with the average monthly surplus (income minus expenses). A goal is rated *Looks achievable* when it needs at most 80% of the surplus, *Possible, but tight* when it needs up to 100%, and *Required savings exceed your surplus* otherwise. In the last case, the app shows the monthly shortfall, the percentage cut in spending needed, and an estimated completion date at the current surplus. It also warns when several active goals together need more than the surplus, and when fewer than three months of data are available.

For example, in Figure 4.4 the goal *CAR* needs ₹4,50,000 more in 7 months, which is ₹64,286 per month. Against an estimated surplus of ₹36,850, this leaves a shortfall of ₹27,436 per month, or about 13 months at the current surplus.

4.6.4 Saving Suggestions

Categories are grouped as *essential* (Bills, Health, Education), *flexible* (Food, Transport) and *discretionary* (Shopping, Entertainment, Other). Suggestions appear after at least five expenses are recorded:

- Up to three non-essential categories averaging at least ₹500 per month and at least 10% of spending get a "trim by 10%" suggestion with the estimated monthly saving.
- An essential category above 25% of spending gets an information note, never a cut.
- A category that has risen by more than 20% and at least ₹500 over last month is flagged.
- A savings rate below 10% (or spending above income) triggers a general savings note.

The user can mark each suggestion as done or dismiss it, and this choice is saved on the server.

4.6.5 Investment Guide

The investment guide is **educational only**. The user chooses an objective, a time horizon and a risk comfort level. The guide then compares seven common options against these inputs: savings account, Fixed Deposit, Recurring Deposit, government-backed schemes, mutual funds, direct equity and gold. Each option is labeled *Fits*, *Partly fits* or *Less aligned*. No interest rates or market returns are shown, nothing is bought for the user, and the guide states that it is not professional financial advice.

4.6.6 Bill Reminder

Each unpaid bill is classified by its due date as *Overdue*, *Due Soon* (within 7 days) or *Upcoming*; paid bills are shown separately. When marking a bill as paid, the user can also record the payment as an expense in the *Bills* category, so that the balance stays correct.

4.6.7 Financial Health Score

The score is out of 100 and is built from the factors in Table 4.5. Factors without enough data are left out rather than counted as zero:

Score = Points Earned ÷ Points Possible (available factors only) × 100

Scores of 80 and above are labeled *Strong*, 60–79 *Good* and 40–59 *Fair*; lower scores are labeled *Needs attention*. No score is shown with fewer than five transactions or no income.

Table 4.5: Financial Health Score factors

Factor	Weight	Rule
Savings rate	30	30 × min(rate ÷ 20%, 1); saving 20% or more earns full points
Spending vs income	25	25 × (1 − expense ratio) ÷ 0.5, limited to 0–25

Factor	Weight	Rule
Savings consistency	20	Share of recorded months with income above expenses (needs 2 months)
Budget adherence	15	Not available: the app has no user-set budgets, so it is left out
Bill payments	10	Share of paid or overdue bills that were paid on time

In Figure 4.6, the user saved 61.4% of income (30/30), spent 38.6% of income (25/25) and saved money in 1 of 2 months (10/20). Budget and bill factors were not available, so the score is 65 ÷ 75 × 100 ≈ **87**.

4.6.8 Monthly PDF Report

For a chosen month, the backend selects only the signed-in user's transactions for that month and the previous month and builds the PDF with ReportLab. The report contains summary figures, a category breakdown with a pie chart, an income-versus-expense chart, plain-language summary sentences compared with the previous month, and the full transaction list.

4.7 TESTING AND RESULTS

The project includes two automated test suites. The frontend suite (finance.test.js, 20 tests, run with *npm test*) checks the calculations: dates and currency format, balance, goal metrics, goal feasibility verdicts, suggestions, bill status and every Health Score rule. The backend suite (test_api.py, 14 tests, run with *python -m unittest test_api -v*) uses a temporary database. It checks sign-up and login validation, token protection, transaction validation, edit and delete, data isolation between users, goals, bills, suggestion states and the PDF report.

Table 4.6: Test results

Test	Expected Result	Result
Frontend calculation tests (20)	All calculations match hand-worked values	Passed (20 of 20)

Test	Expected Result	Result
Backend API tests (14)	Validation, security and CRUD behave as specified	To be recorded after running
Sign in with correct details	Dashboard opens with user's name	Observed (Fig. 4.3)
Add income and expenses	Totals and savings update correctly	Observed (Fig. 4.3)
Goal needing more than surplus	Shortfall and options shown	Observed (Fig. 4.4)
Bill due within 7 days	Listed as Due Soon	Observed (Fig. 4.5)
Discretionary spending recorded	10% trim suggestions with savings	Observed (Fig. 4.7)
Two months of history	Score with factor breakdown	Observed (Fig. 4.6)
Download monthly report	PDF with that month's data	To be recorded after testing

No response-time benchmark or user survey was carried out, so no quantitative performance results are claimed. The quality of the insights depends on the accuracy and completeness of the manually entered data.

4.8 SCREENSHOTS

The screenshots below were captured from the running application using demonstration data entered by the team. Some labels changed slightly in the final build (for example, the top Health Score band is now *Strong*).

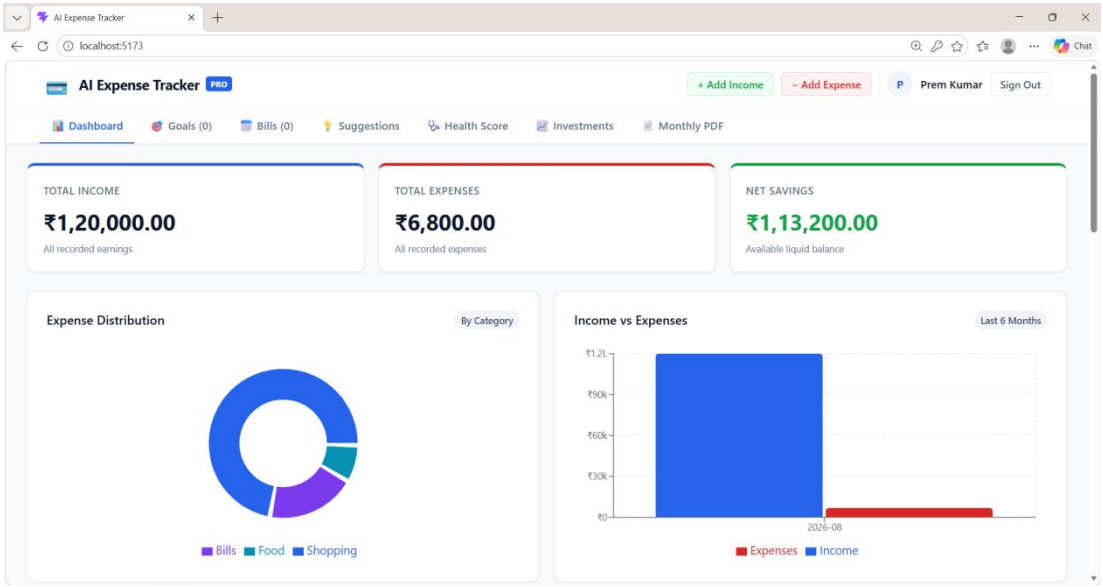


Figure 4.3: Dashboard with summary cards and charts

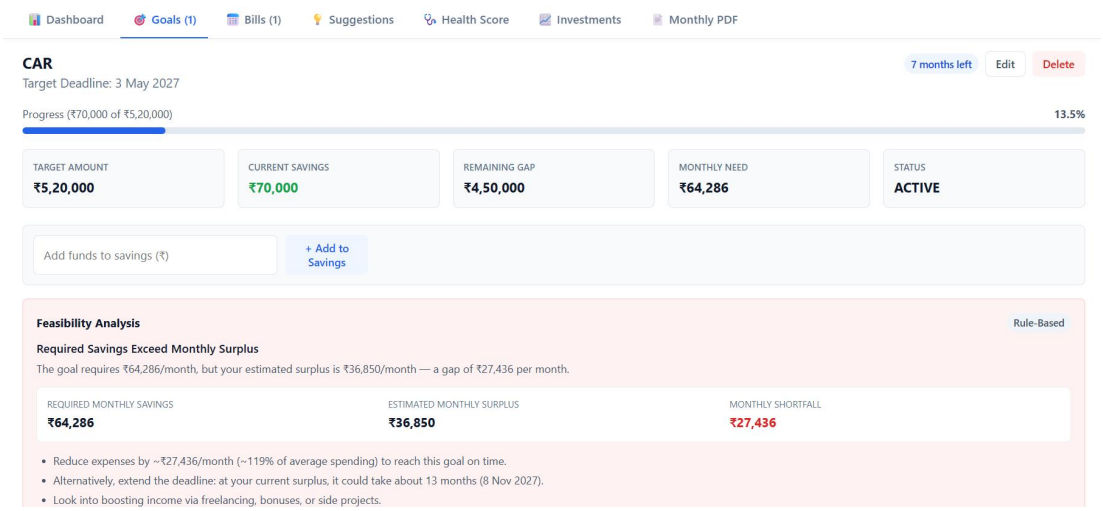


Figure 4.4: Financial Goal Planner with feasibility analysis

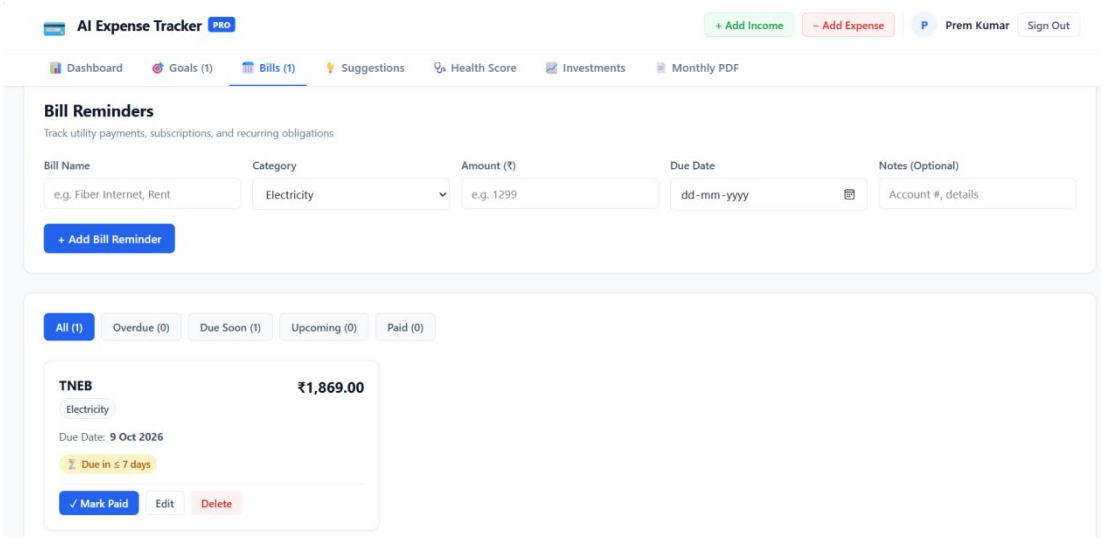


Figure 4.5: Bill Reminder module

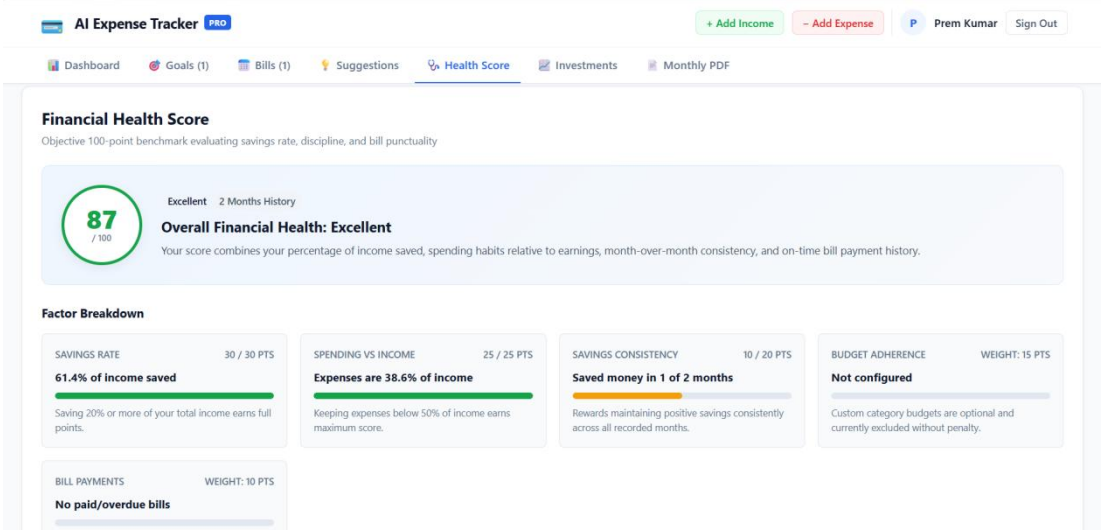


Figure 4.6: Financial Health Score

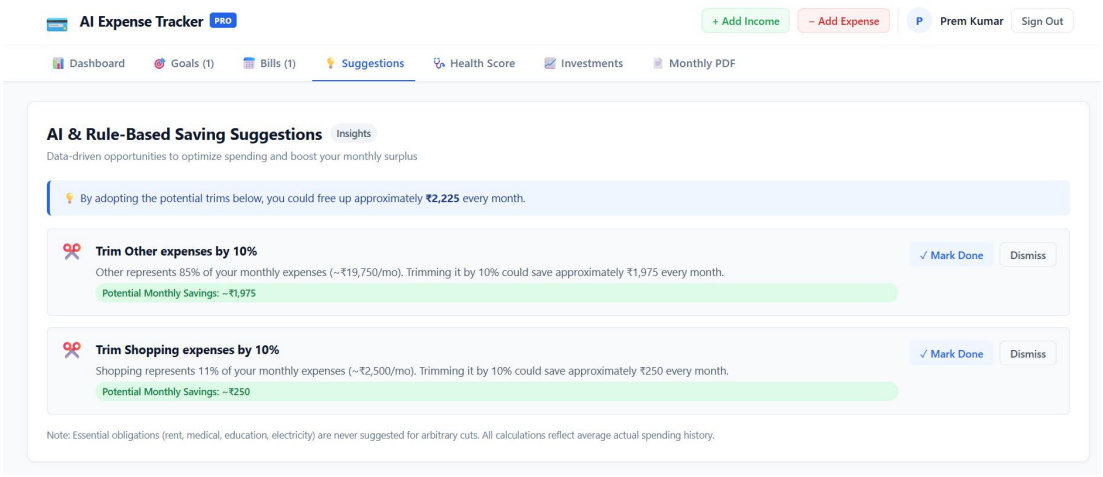


Figure 4.7: Saving Suggestions

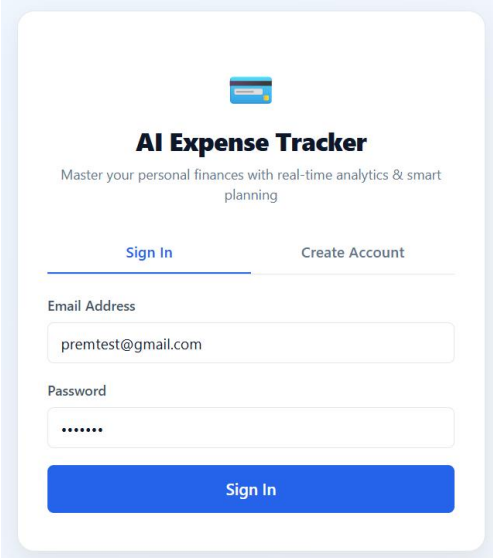


Figure 4.8: Login page

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

The AI-Based Expense Tracker with Analytics and Investment Recommendations was developed to make everyday money management simpler and better informed. The project delivers a working web application built with React.js, Python Flask and SQLite. Users record their transactions manually and receive analytics and guidance calculated from their own data.

The main achievements are:

- secure accounts with hashed passwords, signed login tokens and strict per-user data isolation;
- verified transaction entry with browser and server validation, a review step, and editing and deletion;
- automatic totals, category and monthly charts, expense prediction and budget recommendations;
- a Goal Planner with transparent feasibility analysis, rule-based saving suggestions, an educational investment guide and bill reminders;
- an explainable 100-point Financial Health Score and a downloadable monthly PDF report;
- an automated test suite for the calculation logic, all 20 of whose tests pass.

Because every recommendation is rule-based and states the figures behind it, users can understand why a suggestion was made, and because no bank connection is required, they stay in control of their data. The project meets its objectives as a student prototype and provides a sound base for the data-driven features below.

5.2 LIMITATIONS

- **Manual data entry:** Missing or incorrect entries directly affect every summary and recommendation.
- **Short histories:** Predictions, the estimated surplus and the Health Score become meaningful only after several months of complete data.
- **Rule-based intelligence:** No trained machine-learning model is used and no prediction accuracy is claimed.
- **No user budgets:** Users cannot yet set their own category budgets, so the Budget Adherence factor of the Health Score is not available.
- **Educational investment guidance only:** No live market data is used, and the guide does not replace professional advice.
- **In-app reminders only, local deployment:** No email or mobile notifications are sent, and the system has not been evaluated with real users or deployed at scale.

5.3 FUTURE ENHANCEMENTS

- **Advanced machine learning:** Train and evaluate models such as the ensemble and LSTM approaches in [1], [3] and [4] on consented, anonymized data.
- **Easier data entry:** Voice-based entry, photo-based bill scanning with OCR, and secure import from bank accounts, each confirmed through the existing verification step.
- **User-set budgets and notifications:** Category budgets with alerts, and email or mobile reminders for bills and goals.
- **AI financial assistant:** A conversational assistant that answers questions about the user's own data.
- **Deployment:** Cloud hosting with a production database, stronger security controls and mobile access.

REFERENCES

[1] P. Chahar, Y. K. Vishwakarma, R. Mishra, and G. Paliwal, "Artificial Intelligence Powered Personal Finance Management System," *SSRN Electronic Journal*, 2025. [Online]. Available: https://ssrn.com/abstract=6377518

[2] R. Jadhav and G. A. Patil, "AI-ML Based Expense Categorization and Budgeting System for Personalized Financial Management," *Communications on Applied Nonlinear Analysis*, vol. 32, no. 10s, pp. 1643–1657, 2025.

[3] J. Kumar, A. Vishwakarma, N. Sainy, A. Kumar, and R. Aggarwal, "Personal Finance Manager with Predictive Analytics Using LSTM for Enhanced Financial Decision Making," *International Journal of Research – GRANTHAALAYAH*, vol. 12, no. 5, pp. 156–169, May 2024.

[4] R. S. Thakur and A. Jadhav, "Expense Tracker Management System Using Machine Learning," *Sigma Journal of Engineering and Natural Sciences*, vol. 43, no. 4, pp. 1265–1275, Aug. 2025.

[5] S. R. Saranya, N. Steevan D'Souza, and A. Mohamed Fahadhu, "AI-Powered Intelligent Expense Tracking and Predictive Financial Management System," *Central Asian Journal of Mathematical Theory and Computer Sciences*, vol. 7, no. 4, pp. 83–97, Oct. 2026.

[6] Meta Open Source, "React — The library for web and native user interfaces," official documentation. [Online]. Available: https://react.dev

[7] Pallets Projects, "Flask Documentation." [Online]. Available: https://flask.palletsprojects.com

[8] SQLite Consortium, "SQLite Documentation." [Online]. Available: https://www.sqlite.org/docs.html

[9] SQLAlchemy Authors, "SQLAlchemy — The Database Toolkit for Python," documentation. [Online]. Available: https://www.sqlalchemy.org

[10] Recharts Group, "Recharts — A composable charting library built on React components." [Online]. Available: https://recharts.org

[11] ReportLab Inc., "ReportLab PDF Library User Guide." [Online]. Available: https://docs.reportlab.com

APPENDIX A. CONFERENCE PAPER

AI-Based Expense Tracker with Analytics and Investment Recommendations

M. Prem Kumar
Dept. of AI & ML
VISTAS, Chennai, Tamil Nadu, India

A. Subash
Dept. of AI & ML
VISTAS, Chennai, Tamil Nadu, India

V. Dhanush Raj
Dept. of AI & ML
VISTAS, Chennai, Tamil Nadu, India

Project Guide: Dr. Meenakshi N., Department of Artificial Intelligence and Machine Learning, Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India

***Abstract*—Managing personal expenses well is key to financial stability, yet many people struggle to keep track of daily spending, understand where their money goes, and plan their savings. This paper presents an AI-based expense tracker with analytics and investment recommendations, a web-based system in which users record income and expenses, categorize transactions, and monitor budgets on an interactive dashboard. Machine learning is applied to past transaction data to identify spending patterns, predict future expenses, and flag overspending. Users can also define financial goals, such as buying a car or funding an education plan, and receive investment suggestions (Fixed Deposits, Mutual Funds, Gold, or Stocks) matched to their income, savings, goal amount, and risk preference. The application uses React.js for the frontend, Flask for the backend, SQLite for persistent storage, and scikit-learn for the analytics engine. By bringing expense tracking, predictive analytics, budget alerts, and goal-based investment guidance together on one platform, the system aims to improve financial literacy and encourage more disciplined personal money management.**

***Index Terms*—artificial intelligence, machine learning, expense tracking, financial analytics, expense prediction, investment recommendation, budget management, goal-based saving.**

I. Introduction

Keeping personal finances in order helps people rein in spending, build the habit of saving, and work toward bigger long-term goals. Day-to-day transactions pile up quickly across food, transport, shopping, education, and bills, so recording and making sense of them by hand takes a lot of time and invites mistakes. Notebooks and spreadsheets leave all the arithmetic and pattern-spotting to the user, and many of today's expense apps stop at storing records, offering little help in understanding how a person actually spends or what they will need next.

In this paper we propose an AI-based expense tracker with analytics and investment recommendations. It is a web application where users enter their income and expenses by hand, assign each transaction a category, and see everything summarized on one dashboard. A machine-learning layer studies the recorded data to find spending patterns, estimate upcoming expenses, and raise overspending alerts. Users can also set specific financial goals and get a suggested investment category based on their income, expenses, savings, goal amount, and risk preference, which takes the tool beyond plain bookkeeping and toward planning around goals.

A. Objectives

The system is designed to meet the following objectives:

- To build an easy-to-use tool for tracking income and expenses.
- To let users enter and categorize their own transactions.
- To study past data and bring out useful spending insights.
- To forecast future expenses from earlier spending habits.
- To warn users when a budget limit is crossed.
- To help users set financial goals and follow their progress.
- To suggest suitable investments using income, savings, goal, and risk preference.

II. Literature Review

Many researchers have looked at how AI and machine learning can support personal finance. Table I lists five recent studies that shaped our design, along with the techniques they used, what they contributed, and the limitations they reported.

Chahar *et al.* developed an AI-based personal finance assistant that uses machine learning and natural language processing to produce customized budget advice and financial education material, and tested it with an early prototype [1]. Jadhav and Patil presented a hybrid machine-learning framework that merges multiple classifiers for categorizing expenses and building budgets; it reached a recommendation accuracy of about 98.5% but is harder to interpret [2]. Kumar *et al.* used Long Short-Term Memory (LSTM) networks on historical time-series data to forecast income, expenses, and savings, obtaining around 85% accuracy, although the model handled irregular, one-off financial events poorly [3]. Thakur and Jadhav tested a range of regression and ensemble models on a household transactions dataset and found that a voting ensemble regressor performed best, with an R^2 of 78.11% and the smallest relative absolute error [4]. Saranya *et al.* paired OCR-based receipt scanning with BERT-based transaction classification and LSTM forecasting, achieving categorization accuracy above 94%, but their approach needs ongoing access to bank, card, and wallet data [5].

TABLE I
SUMMARY OF RELATED WORK

Ref.	Study (Author, Year)	AI/ML Technique Used	Key Contribution	Reported Limitation
[1]	Chahar *et al.*, 2025	ML/NLP-based predictive analytics, recommendation models	AI assistant offering customized budget advice and financial education	Performs poorly on irregular or one-off expenses
[2]	Jadhav and Patil, 2025	Hybrid ML (Decision Tree, SVM, Random Forest, ANN)	Automatic expense categorization and budgeting, ~98.5% recommendation accuracy	Needs large labeled datasets; hybrid model is hard to explain
[3]	Kumar *et al.*, 2024	LSTM (recurrent neural network)	Predicts income, expenses, and savings from past time-series data	Weak on irregular one-time income or expenses (bonuses, emergencies)
[4]	Thakur and Jadhav, 2025	XGBoost, Random Forest, SVM, MLP, Voting Ensemble	Voting ensemble regressor for future expenses ($R^2 = 78.11\%$)	Relies on a generic household dataset rather than per-user data
[5]	Saranya *et al.*, 2026	OCR + BERT (classification) + LSTM (forecasting)	Automated receipt scanning and categorization (>94% accuracy, ±6% forecast variance)	Needs OCR infrastructure and access to sensitive bank/wallet data

III. Research Gap

The reviewed studies leave several gaps that motivate this work:

- Several systems depend on direct bank or card integration, which raises privacy and security concerns for users who would rather keep their financial data under their own manual, offline-first control.
- Recommendations are often based on spending history alone, without weighing income, savings, a particular goal amount, and the user's risk appetite together.
- Where goal features exist, they usually just track progress and do not convert the goal into a concrete monthly saving or investment plan.
- Models trained on generic or simulated household datasets may not fit an individual's irregular income and spending.
- Few systems separate essential everyday spending from unnecessary or excessive spending in a way users can act on right away.

Our system tackles these gaps with a manual-entry-first design that does not require bank integration, together with an analytics engine that considers income, expenses, savings, goal amount, and risk preference at once to give goal-based investment suggestions alongside regular budget monitoring.

IV. Proposed System

The proposed system is a web application in which users enter income and expense transactions by hand, each with an amount, category, description, and date. The backend checks every entry before saving it, and the analytics engine then refreshes the spending summaries, forecasts, and budget status. Users can also define a financial goal (a target amount and time frame); the system checks whether it is achievable and suggests an investment category.

A. System Architecture

Fig. 1 shows the modular architecture. The User Data Input Module collects the income, expenses, goals, and preferences that the user enters manually. The Frontend Interface Module, built in React, contains the login/sign-up flow, the dashboard, the income and expense screens, analytics views, the goal planner, and the investment-suggestion display. The Core Backend Processing Module, written in Flask, handles authentication, income and expense processing, goal management, financial calculations, and API management. Data is saved in the Database Storage Module (SQLite), which holds users, income, expenses, goals, bills, and preferences. The AI/ML Engine Module works on this stored data for spending-pattern analysis, expense prediction, budget recommendation, goal-feasibility analysis, and investment-category suggestion, and the Results and Insights Delivery Module presents its output as dashboards, analytics, and recommendations. A Security and Data Protection layer, which includes user authentication, role-based access control, data encryption, secure API access, and regular backups, cuts across all modules.

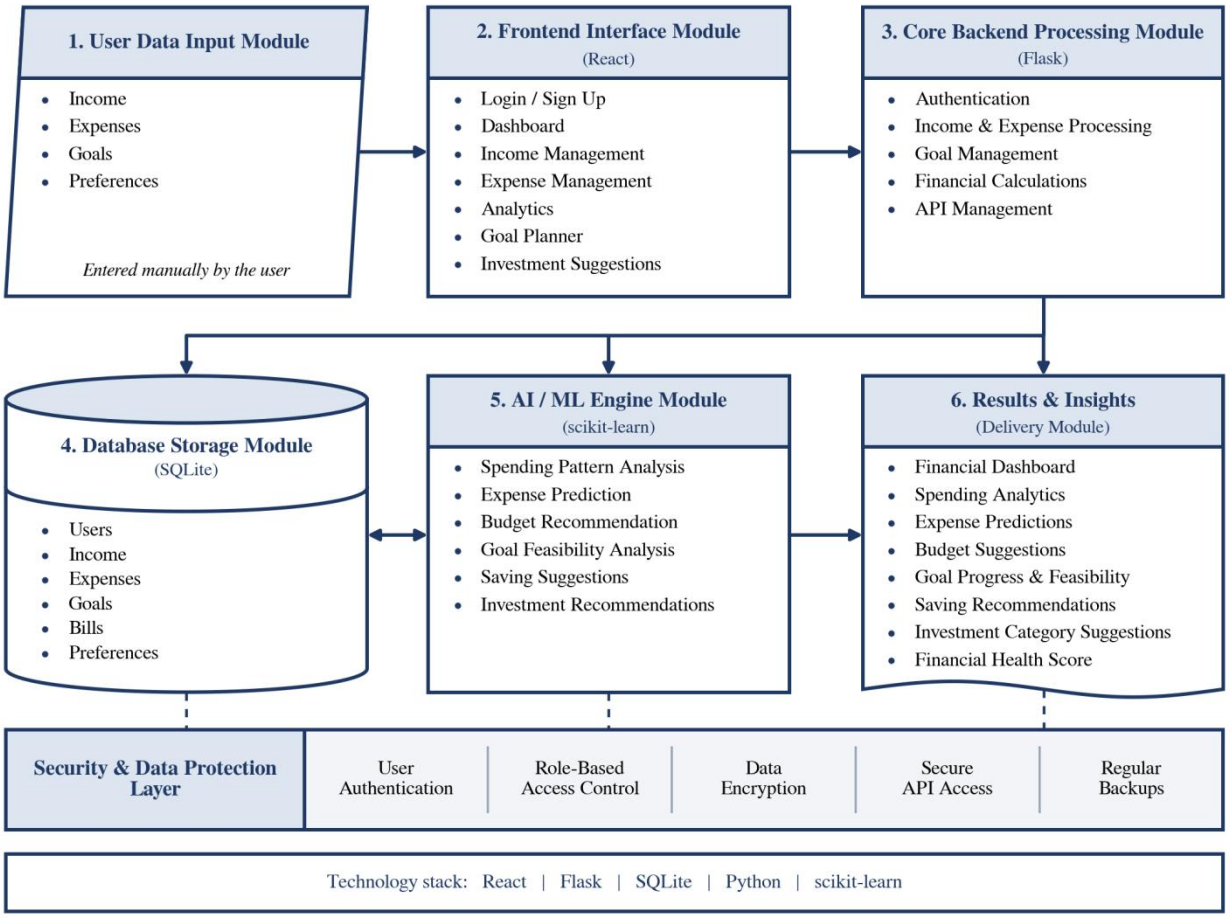


Fig. 1. System architecture of the AI-based expense tracker.

B. System Workflow

Fig. 2 shows the workflow followed for each income or expense entry. Once the user logs in and submits a transaction, the system validates it; an invalid entry is rejected with an on-screen error, while a valid one is saved to the database. The AI/ML engine then categorizes the transaction, updates the spending-pattern analysis, predicts expenses for the near future, and recalculates savings. If the projected expenses in a category go beyond the budget limit the user has set, an overspending alert is raised; if not, the system moves on to check goal feasibility and, where relevant, suggests an investment category. Finally, the dashboard is refreshed with charts, budget status, and savings progress for the user to review.

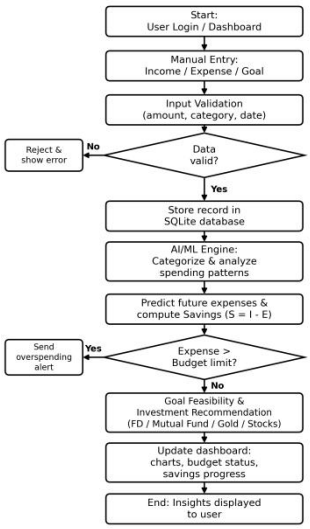


Fig. 2. Expense tracking and recommendation workflow.

V. METHODOLOGY

A. User Authentication

Registration and login keep each user's financial records tied to their own account. Passwords are stored with a secure hashing method, and session-based access control decides which pages and API endpoints a logged-in user can reach.

B. Income and Expense Entry

Each transaction is entered through a form that asks for the amount, a category (for example food, transportation, shopping, education, or bills), an optional description, and a date. Common spending areas come as predefined categories, and the design also lets users add their own.

C. Data Validation

Before a transaction is saved, the backend confirms that all required fields are filled in, that the amount is a positive number, and that the chosen category and transaction type are valid. Entries that fail these checks are rejected and the user is asked to correct them, which lowers the chance of corrupted or inconsistent records reaching the database.

D. Financial Analytics

Given a set of income transactions I_i and expense transactions E_j recorded for a user, the system computes three core quantities: total income (1), total expenses (2), and total savings (3).

$$I = \Sigma\, I_i, \quad i = 1, \ldots, n \tag{1}$$

$$E = \Sigma\, E_j, \quad j = 1, \ldots, m \tag{2}$$

$$S = I - E \tag{3}$$

Here, I_i is the i-th income transaction, E_j is the j-th expense transaction, n is the number of income records, and m is the number of expense records. Together with a category-wise breakdown of E, these figures appear on the dashboard so users can see where most of their money goes.

E. Budget Monitoring

For each expense category c, the user may optionally set a budget limit $B(c)$. Given the actual expense $E(c)$ logged in that category during the monitoring period, the system applies the rule

$$R(c) = \begin{cases} \text{“Review budget”}, & \text{if } E(c) > B(c) \\ \text{“Within budget”}, & \text{if } E(c) \le B(c) \end{cases} \tag{4}$$

This rule marks any category whose spending has crossed the user-defined limit and triggers an overspending alert on the dashboard.

F. Goal-Based Investment Recommendation

Suppose a user sets a financial goal with target amount G and a time horizon of N months, and has current savings of C_s, the required monthly saving M is estimated as

$$M = (G - C_s) / N \tag{5}$$

The system compares M against the user’s average historical monthly savings to judge whether the goal is feasible. It then maps the goal’s time horizon and the user’s stated risk preference (low, medium, or high) to an investment category: Fixed Deposits and Gold for short-horizon, low-risk goals; Mutual Funds for medium-term, moderate-risk goals; and Stocks for longer-horizon, higher-risk goals. The result is a concrete, goal-linked starting point for further financial decisions instead of generic investment advice.

VI. System Implementation

The application follows a client–server architecture. Table II lists the technologies used to build each layer of the system.

TABLE II
TECHNOLOGY STACK

Component	Technology	Purpose
Frontend	React.js, HTML, CSS, JavaScript	Login/sign-up, dashboard, transaction forms, analytics and goal-planner views
Backend	Python, Flask	Authentication, transaction processing, financial calculations, REST API
Database	SQLite	Storage of users, income, expenses, goals, bills, and preferences
Analytics engine	Python, scikit-learn	Spending-pattern analysis, expense prediction, recommendation logic

Development was done in Visual Studio Code, with Node.js and npm for frontend tooling, on a Windows 10/11 machine with at least 4 GB of RAM (8 GB recommended) and 10 GB of free storage. Figs. 3 and 4 show the implemented dashboard and the expense-entry form, respectively.

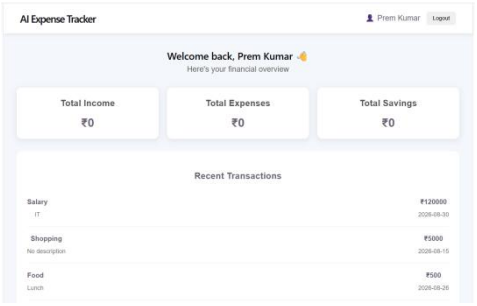


Fig. 3. Dashboard showing income, expenses, savings, and recent transactions.

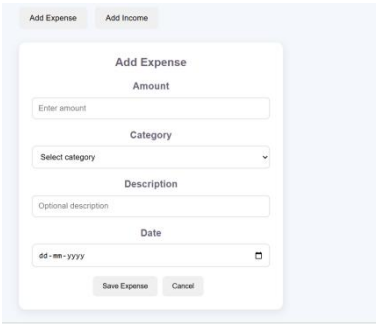


Fig. 4. Expense-entry form with amount, category, description, and date fields.

VII. Results and Discussion

The application is currently at the first-review stage of implementation, with authentication, the dashboard, and manual transaction entry working. A full quantitative evaluation of the prediction and recommendation modules is therefore left for the next phase. Evaluation will follow the criteria in Table III, in line with the approach taken in related work [1], [4].

TABLE III
PLANNED EVALUATION CRITERIA

Evaluation Parameter	Testing Approach
Authentication	Test valid and invalid login credentials
Transaction validation	Submit valid, incomplete, and invalid transactions
Financial calculations	Compare dashboard totals (I, E, S) against manually computed values
Expense prediction	Compare predicted and actual expense using MAE and R^2, following [3], [4]
Budget alerts	Test alert triggering across categories with known over- and under-budget scenarios
Investment recommendations	Test output across varied goal, savings, and risk-preference inputs

No numerical results are claimed at this stage. Measured accuracy, error, and user-satisfaction figures will be reported after model training and user testing are finished.

VIII. Advantages of the Proposed System

- Centralized, manual-entry-first handling of personal financial records with no mandatory bank integration.
- Automatic calculation of income, expenses, and savings, so users avoid manual arithmetic.
- Validated data entry that cuts down incomplete or invalid records.
- Category-wise spending charts that show the main expense areas.
- Overspending alerts driven by user-defined budget limits.
- Investment category suggestions tied to the user's goal instead of generic advice.
- Lightweight, open-source technology stack (React, Flask, SQLite, scikit-learn).

IX. Limitations and Future Work

Since transactions are entered by hand, the accuracy of the analytics and recommendations depends on how complete and correct the user's input is; missing entries can skew spending summaries. The budget and goal-feasibility logic is currently rule-based and does not yet use models learned from the user's own history. Planned enhancements include:

- Category-specific expense-prediction models (e.g., regression or LSTM-based, as in [3], [4]) trained on each user's own transaction history.
- Optional OCR-based receipt scanning to cut down manual entry, following the approach in [5].
- Interactive drill-down charts for monthly and category-wise views.
- Goal progress tracking with visual milestones and reminders.
- Stronger authentication (e.g., multi-factor) and encryption of stored financial data.
- Deployment as a hosted, multi-device application.

X. Conclusion

This paper described an AI-based expense tracker with analytics and investment recommendations that brings manual transaction entry, data validation, financial analytics, budget monitoring, and goal-based investment suggestions together in one web application built on React, Flask, SQLite, and scikit-learn. Unlike earlier systems that depend on bank-account integration or look at spending history alone, our design leaves data entry in the user's hands while reasoning over income, expenses, savings, goals, and risk preference together to give practical, goal-linked guidance. The architecture and workflow presented here lay the groundwork for the prediction and recommendation modules now under development, and evaluation against the criteria in Table III is the next milestone.

References

[1] P. Chahar, Y. K. Vishwakarma, R. Mishra, and G. Paliwal, "Artificial intelligence powered personal finance management system," *SSRN Electronic Journal*, 2025. [Online]. Available: https://ssrn.com/abstract=6377518

[2] R. Jadhav and G. A. Patil, "AI-ML based expense categorization and budgeting system for personalized financial management," *Communications on Applied Nonlinear Analysis*, vol. 32, no. 10s, pp. 1643–1657, 2025.

[3] J. Kumar, A. Vishwakarma, N. Sainy, A. Kumar, and R. Aggarwal, "Personal finance manager with predictive analytics using LSTM for enhanced financial decision making," *International Journal of Research – GRANTHAALAYAH*, vol. 12, no. 5, pp. 156–169, May 2024.

[4] R. S. Thakur and A. Jadhav, "Expense tracker management system using machine learning," *Sigma Journal of Engineering and Natural Sciences*, vol. 43, no. 4, pp. 1265–1275, Aug. 2025.

[5] S. R. Saranya, N. Steevan D'Souza, and A. Mohamed Fahadhu, "AI-powered intelligent expense tracking and predictive financial management system," *Central Asian Journal of Mathematical Theory and Computer Sciences*, vol. 7, no. 4, pp. 83–97, Oct. 2026.

APPENDIX B. SOURCE CODE

Only the key excerpts of the source code are given below. Repeated, styling and supporting code has been left out to keep this appendix short; the complete code is available in the project repository.

B.1 Running the application

```
cd AI-Expense-Tracker\backend
venv\Scripts\activate
python app.py                       (backend on http://127.0.0.1:5000)

cd AI-Expense-Tracker\frontend
npm install
npm run dev                         (open the local URL shown)
```

B.2 backend/models.py

```
from flask sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class User(db.Model):
    id = db.Column(db.Integer, primary key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(255), nullable=False)      # hashed

class Transaction(db.Model):
    id = db.Column(db.Integer, primary key=True)
    user id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False)
    type = db.Column(db.String(20), nullable=False)           # income / expense
    amount = db.Column(db.Float, nullable=False)
    category = db.Column(db.String(50), nullable=False)
    description = db.Column(db.String(200))
    date = db.Column(db.String(20), nullable=False)           # YYYY-MM-DD
```

B.3 backend/app.py (key parts: authentication and adding a transaction)

```
import math, os, re
from datetime import date, datetime
from functools import wraps
from flask import Flask, g, jsonify, request, send file
from flask cors import CORS
from itsdangerous import BadSignature, SignatureExpired, URLSafeTimedSerializer
from werkzeug.security import check password hash, generate password hash
from models import Bill, Goal, SuggestionState, Transaction, User, db
from report import build monthly report
def login required(view):
    """The route runs only when a valid login token is sent."""
    @wraps(view)
    def wrapper(*args, **kwargs):
        header = request.headers.get("Authorization", "")
        if not header.startswith("Bearer "):
            raise ApiError("Please log in to continue.", 401)
        try:
            payload = token serializer.loads(header[7:], max age=TOKEN MAX AGE SECONDS)
        except (SignatureExpired, BadSignature):
            raise ApiError("Invalid or expired session. Please log in again.", 401)
        g.user = db.session.get(User, payload.get("uid"))
        if g.user is None:
```

```
            raise ApiError("Account not found. Please log in again.", 401)
        return view(*args, **kwargs)
    return wrapper

@app.route("/login", methods=["POST"])
def login():
    data = json body()
    email = str(data.get("email") or "").strip().lower()
    user = User.query.filter(db.func.lower(User.email) == email).first()
    password = str(data.get("password") or "")
    if user is None or not check password hash(user.password, password):
        raise ApiError("Invalid email or password", 401)
    token = token serializer.dumps({"uid": user.id})
    user_info = {"id": user.id, "name": user.name, "email": user.email}
    return jsonify({"user": user info, "token": token}), 200

def read transaction fields(data, tx type):
    amount = parse amount(data.get("amount"))
    category = clean_text(data.get("category"), "Category", 50, required=True)
    allowed = EXPENSE CATEGORIES if tx type == "expense" else INCOME CATEGORIES
    if category not in allowed:
        raise ApiError(f"'{category}' is not a valid {tx type} category.")
    description = clean_text(data.get("description"), "Description", 200)
    return amount, category, description, parse date(data.get("date"))

def create transaction(tx type):
    amount, category, description, tx date = read transaction fields(json body(), tx type)
    db.session.add(Transaction(user_id=g.user.id, type=tx_type, amount=amount,
                               category=category, description=description, date=tx date))
    db.session.commit()
    return jsonify({"message": f"{tx type.capitalize()} added successfully"}), 201

@app.route("/expense", methods=["POST"])
@login required
def add expense():
    return create_transaction("expense")
```

B.4 frontend/src/utils/finance.js (goal calculation)

```
const clamp = (value, min, max) => Math.min(max, Math.max(min, value));
export const monthKey = (isoDate) => String(isoDate || "").slice(0, 7);

/* Goal maths: progress, remaining amount and monthly saving required. */
export function computeGoalMetrics(goal, today = todayISO()) {
  const target = Number(goal.target amount) || 0;
  const saved = Number(goal.current savings) || 0;
  const remaining = Math.max(0, target - saved);
  const progress = target > 0 ? clamp((saved / target) * 100, 0, 100) : 0;
  const completed = target > 0 && saved >= target;
  const daysLeft = daysBetween(today, goal.target date);
  const overdue = !completed && daysLeft < 0;
  const monthsLeft = daysLeft >= 0 ? Math.max(1, Math.ceil(daysLeft / 30.4375)) : 0;
  const monthlyRequired = completed ? 0 : overdue ? null : remaining / monthsLeft;
  return { target, saved, remaining, progress, completed, overdue,
           monthsLeft, monthlyRequired };
}
```